

Appunti 20 Feb 2026

In Angular, un servizio rappresenta la parte logica e invisibile di un'applicazione. Può contenere sia logica di business che stato condiviso tra più componenti. In pratica, mentre i componenti si occupano della parte visiva – cioè dell'interfaccia utente e dell'interazione con l'utente – i servizi gestiscono tutto ciò che riguarda i dati, la comunicazione con l'esterno e le operazioni di supporto.

Un esempio concreto è il `ExamParameterService`. Questo servizio fornisce due metodi principali:

`insert(parameter)`, che serve per inserire un nuovo parametro,

`parameters()`, che restituisce la lista di tutti i parametri disponibili.

Per effettuare chiamate a un backend o a delle API esterne, Angular mette a disposizione l'oggetto `HttpClient`, che viene usato proprio all'interno dei servizi per gestire la comunicazione HTTP.

Un aspetto fondamentale è che i servizi in Angular non vengono istanziati manualmente, ma iniettati tramite il meccanismo di Dependency Injection. Quando un servizio viene dichiarato (ad esempio con `providedIn: 'root'`), Angular lo registra nel context globale dell'applicazione, rendendolo disponibile ovunque serva senza doverlo ricreare. Questo approccio garantisce che esista una sola istanza condivisa, semplificando la gestione dello stato e migliorando le prestazioni.

In Angular, i servizi hanno un ruolo fondamentale perché rappresentano il cuore logico dell'applicazione. Sono il luogo in cui si concentra tutta la logica di business, la gestione dei dati e delle chiamate esterne.

Un principio importante è che le richieste HTTP devono essere eseguite solo nei servizi. Anche se Angular tecnicamente permette di farle direttamente nei componenti tramite `HttpClient`, questa pratica è fortemente sconsigliata. In produzione non si fa mai – è considerato bad practice e, di fatto, vietato per motivi di organizzazione, manutenzione e riusabilità del codice.

Il motivo è semplice: i componenti dovrebbero limitarsi a gestire la parte visiva e l'interazione con l'utente, senza occuparsi della logica di comunicazione con il backend. I servizi, invece, si occupano di ricevere, elaborare e fornire i dati pronti da mostrare nell'interfaccia, mantenendo così una chiara separazione dei ruoli e un codice più pulito e riutilizzabile.

Un signal in Angular è una funzione che avvolge un valore e lo rende reattivo. Tiene dentro uno stato; quando quel valore cambia (con `set`, `update`, ecc.), tutti i signal “derivati” che lo usano (ad esempio `computed`) possono essere ricalcolati automaticamente, così come la parte di template che lo legge. Quando definiamo un signal privato nel servizio: `private _examParameters = signal<ExamParameters[]>([])`; questo diventa il nostro contenitore scrivibile (`WritableSignal`) che terrà i dati dal backend. Iniziamo con un array vuoto `[]` come valore di default, ma quando arriva la risposta HTTP lo aggiorniamo con `set()` o `update()`. Il resto del mondo (componenti, altri servizi) non può scriverlo direttamente – solo il servizio controlla quando modificare `_examParameters`. Questo è il principio di incapsulamento puro.

Poi esponiamo la versione pubblica e readonly: `public readonly examParameters = this._examParameters.asReadOnly();` `asReadOnly()` crea un signal di sola lettura (`ReadOnlySignal`) che puntatore allo stesso signal originale.

Chiunque può leggere i dati chiamando `examParameters()`, ma non può modificarli (nessun metodo `set()` o `update()` disponibile).

È un riferimento, non una copia: se il signal originale cambia, anche la versione readonly si aggiorna automaticamente.

Un computed è un signal derivato che calcola il suo valore automaticamente basandosi su altri signals.

In js:

Quando definiamo le eccezioni:

Non esiste `throw new Exception()` come in Java

JavaScript accetta qualsiasi valore (stringa, numero, oggetto, array...)

Il `catch` riceve sempre quello che hai lanciato, senza controllo di tipo